ptbr-writer: português técnico controlado para pessoas e agentes

Escrita agêntica em português brasileiro é uma abstração barata

Matheus Hoffmann ORCID
Escola Politécnica, Universidade Federal do Rio de Janeiro

Rob Kler Soares da Silva Junior ORCID

Outubro de 2026, sem revisão por pares · v0.3.0, preprint · rev. 3e36991+122





Resumo. O ptbr-writer reescreve texto técnico em português brasileiro para uma leitura única: sujeito explícito, uma proposição por frase e termos consistentes. A semente é o Português Técnico Controlado (PTC), de kayquer: uma skill do Claude Code com regras de linguagem controlada em português para um leitor que pode ser um modelo. O ptbr-writer adapta as 8 regras do PTC e acrescenta 5, num total de 13 regras, PTW-1 a PTW-13. Cada pedido passa por um de 5 níveis de rigor, de `estrito` a `smart-note`. Para texto com fontes, a skill `cientifico` acrescenta 12 regras, CIE-1 a CIE-12. As mesmas regras chegam por três caminhos: um plugin do Claude Code, um instalador para o OpenCode e uma extensão de navegador para Chrome e Firefox. A extensão trata a revisão como uma decisão. Além de modelos que geram texto (Claude, DeepSeek ou um modelo local pelo Ollama), ela aceita um modelo de decisão local, servido pelo Ollaya, que diz qual regra cada frase quebra, com uma probabilidade, sem gerar texto. As ferramentas de execução são binários Rust estáticos, fixados por sha256, e cada afirmação não conferida leva a marca ⚠. Este artigo descreve as regras, os níveis, a arquitetura e a política de evidência. Ele não relata medida de acerto: os casos de regressão em `evals/` estão escritos e não foram rodados, e a latência e o acerto dos modelos locais em pt-BR não foram medidos. O código está sob a MPL-2.0 desde a v0.3.0.

Palavras-chave: escrita técnica; português brasileiro; linguagem controlada; agentes de programação; modelos de decisão; reprodutibilidade.

1 Introdução

Um runbook é lido por quem executa o procedimento e, cada vez mais, por um agente que o executa. Nenhum dos dois pode parar para perguntar “você quis dizer X ou Y?”. O PTC, de kayquer, escreveu essa exigência como uma skill do Claude Code, com 8 regras para o português do Brasil [1]. O ptbr-writer parte dessa semente.

A tese deste artigo é que escrita agêntica em português brasileiro é uma abstração barata. Barata porque a abstração inteira é texto: um arquivo de regras em Markdown, que qualquer agente que lê skills carrega, e um binário pequeno que decide o que perguntar antes de escrever. Não há modelo treinado, ajuste fino nem servidor. O artigo sustenta a tese com três pilares: o português brasileiro sob medida (seção 2), a portabilidade (seção 3) e um harness que melhora a si mesmo (seção 4). A seção 5 mostra o que se mediu de custo, e a seção 6, o que não se mediu.

2 Português brasileiro sob medida

O PTC observa que o inglês obriga o sujeito, e por isso o ASD-STE100, a linguagem controlada da indústria aeroespacial, nunca precisou de uma regra para ele [1]. O português permite omitir o sujeito. A primeira regra do ptbr-writer, PTW-1, pede o sujeito explícito e evita o `-se` apassivador, `o mesmo` e os clíticos ambíguos de terceira pessoa (`SKILL.md`). Uma frase como "deve-se verificar os logs" vira "verifique os logs".

As regras PTW-1 a PTW-8 adaptam as 8 do PTC: uma proposição por frase, modalidade e quantidade explícitas, verbo pleno, sintaxe plana, termos congelados, números com unidade e a ortografia do Acordo de 1990, atenta a prefixos técnicos como `microsserviço`. As regras PTW-9 a PTW-13 acrescentam uma política de evidência: toda afirmação tem marca, nenhuma referência é inventada, número tem unidade e origem, não há alegação de pioneirismo e fato fica separado de análise.

O rigor muda com o leitor. Os níveis são `estrito` (runbooks e saídas para agentes), `descritivo` (documentação de sistema), `leve` (comunicados), `científico` (texto com fontes) e `smart-note` (notas para um segundo cérebro). O mesmo pedido pode sair em pt-BR e em ASD-STE100, um texto por idioma. Todo texto que o plugin gera traz o tempo de leitura, calculado pelo binário, com 238 palavras por minuto [2]. Essa taxa é a do inglês; o pt-BR usa a mesma, sem escala ⚠.

3 Portabilidade

As regras ficam num arquivo, `SKILL.md`, com 807 palavras (`wc -w`), e a skill `cientifico` tem 1 087. Um agente que lê skills carrega esses arquivos sem adaptação. O ptbr-writer chega por três caminhos:

- **Claude Code:** um plugin, instalado com um comando e fixado por tag. Um repositório fixa a versão no próprio `.claude/settings.json`, e todos que o abrem usam a mesma.
- **OpenCode:** um script copia as skills como arquivos e instala um plugin que faz o que o hook do Claude Code faz. Ele foi testado no OpenCode 1.19.0: as skills carregam e a instrução chega ao modelo. Que o modelo responda com a ferramenta `question` não foi medido ⚠.
- **Navegador:** uma extensão para Chrome e Firefox 128+ sublinha no campo de texto o que o nível pede para mudar. Nada muda sem o clique em **Aceitar**.

O que roda na máquina é um binário Rust estático, `ptbr`, com 570 112 bytes em `linux-x86_64`, e um par para `linux-aarch64`. O `tools.lock` fixa o commit do picasso que o compila e o sha256 de cada binário. Não há Python nem Node no tempo de execução. O `.pdf` sai do pandoc da máquina ou, sem ele, de um binário que o Nix compila a partir de um commit fixo; a primeira compilação, num runner novo do GitHub com 4 núcleos, levou 892 s.

A extensão leva a portabilidade ao modelo. Ela fala com Claude e DeepSeek, com a chave do usuário, e com dois servidores locais, sem chave. O Ollama serve um modelo que escreve a troca. O Ollaya serve modelos de decisão, que respondem perguntas fechadas com probabilidades e nunca geram texto [3]. Para a extensão, apontar uma regra quebrada é uma decisão: para cada frase, a pergunta é "qual regra esta frase quebra?", com `nenhuma` e as regras do nível como respostas. Os dois servidores também andam juntos: o Ollaya escolhe o tom da

conversa (conciso, socrático ou amigável), e o Ollama escreve nesse tom. Com o Ollaya fora do ar, o Ollama escreve sozinho.

4 Um harness que melhora a si mesmo

Neste artigo, RSI tem um sentido só, o operacional que o repositório usa: o rótulo `rsi` marca "um erro que o harness deixou passar; a correção muda o loop para que ele não volte" (`ISSUE_LABELS.md`). O loop é o `just quality`, que o CI roda em cada pull request. Quatro casos de outubro de 2026 mostram o padrão, todos com o próprio agente como autor do erro:

1. Uma edição pôs duas linhas do roadmap no topo do README, fora da tabela. O `just quality` passou, porque só conferia se o nome de cada peça aparecia em algum lugar. A correção acrescentou uma checagem de estrutura: nenhuma linha de tabela antes do primeiro título.
2. O README chegou a ~31 minutos de leitura. Ele foi cortado para ~8, e o `just quality` passou a falhar acima de 10, pelo mesmo binário que mede o texto do usuário.
3. O README em inglês mostrou a figura com rótulos em português. Cada figura ganhou um gêmeo em inglês, e a checagem falha se um README mostra a figura do outro idioma.
4. A receita de release abriu um editor num shell sem terminal e parou no meio. Sem terminal, ela agora não abre o editor e diz o que fazer.

Em cada caso, a correção do arquivo veio com uma checagem nova, e a checagem é o que fica. O projeto não mede se esse loop reduz erros ao longo do tempo ⚠.

5 O que se mediu de custo

Três pedidos reais ao plugin, com o modelo `claude-opus-5-5`, no commit `ca0e859` do plugin, em 21 de outubro de 2026, foram medidos pelo `claude -p --output-format json` (`docs/eli5/RESULTS.tsv`):

Pedido	Nível	Tokens no total	Lidos do cache	Saída	Custo (US$)
runbook de deploy	estrito	350 529	301 876	8 184	0,5477
comunicado	leve	184 688	155 591	2 401	0,2927
explicar a uma criança	descritivo	232 366	200 269	3 434	0,3380

Entre 84% e 86% dos tokens de cada pedido foram lidos do cache. A abstração em si não tem custo de treino nem de servidor. O custo por pedido é o do modelo que o usuário escolhe. Com um modelo de decisão local, não há cobrança por pedido. No Ollaya Bench, numa CPU de 24 núcleos, o `laya:multilingual` respondeu cinco perguntas em 515,5 ms e o `decima` em 1 212,5 ms, em tarefas em inglês e alemão (`docs/SURVEY_DECISION_MODEL.md`). Esses números não foram medidos em pt-BR ⚠.

6 Limitações

Os casos de regressão em `evals/` estão escritos e não foram rodados por um harness, então o acerto das regras não foi medido. A extensão foi testada à mão no WhatsApp Web, com uma chave real; Gmail e GitHub não foram testados com o editor de verdade. A latência e o acerto do Ollama e do Ollaya em pt-BR não foram medidos, e o limiar de 50% do Ollaya não foi ajustado. Os quatro casos da seção 4 são de um projeto e de um mês. Os três pedidos da seção 5 são uma amostra de três, com um modelo só. A tag v0.2.0 nunca foi criada; a v0.3.0 é a primeira tag desde a v0.1.0.

7 Referências

1. kayquer. *Português Técnico Controlado (PTC)*. Skill do Claude Code, licença MIT. https://github.com/kayquer/portugues-tecnico-controlado/tree/cfdffad0ed98cdebfb7f591481889090a4f4e65b, commit `cfdffad`, lido em 22 de outubro de 2026.
2. Brysbaert, M. (2019). How many words do we read per minute? A review and meta-analysis of reading rate. *Journal of Memory and Language*, 109, 104047. https://doi.org/10.1016/j.jml.2019.104047
3. ollaya-dev. *Ollaya*. Licença Apache-2.0. https://github.com/ollaya-dev/ollaya/tree/39619accce0f5836dbeb60d2b14f5a056c2459ce, commit `39619ac`, lido em 21 de outubro de 2026.

O código, as regras e este texto estão em https://github.com/h0ffmann/ptbr-writer, sob a MPL-2.0. O modelo deste PDF vem do Paper Fabric de h0ffmann/closed-source, commit `563c339` (MIT).